

Machine Learning

MY CHOICE

Date

Page

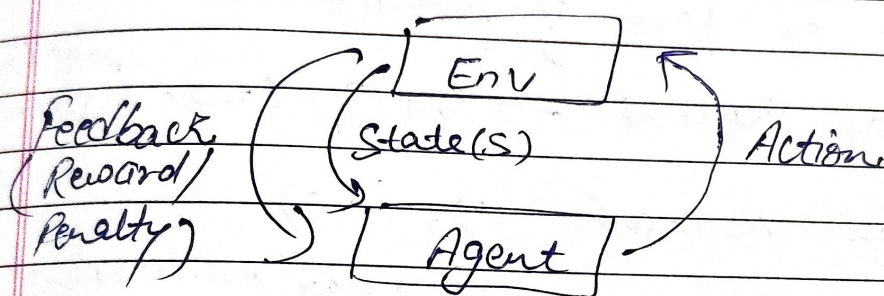
REINFORCEMENT LEARNING

2013, DEEPMIND

↓
RL + DL = BOOM
(Deep-Q)

2016, DEEPMIND → ALPHA-GO

Lee Sedol defeated 4-1-34



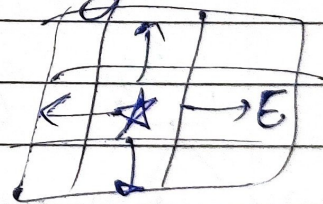
→ Components

- ① $S_t A_t \rightarrow R_{t+1}$ or $R_{t, \infty}$
- ② Policy (π) → Strategy (xxx to win game)
- ③ $G \rightarrow$ Return (future Reward)

⇒ Policy → konsa action le to win the game.

 $S \rightarrow A$

$\pi\left(\frac{A}{S}\right)$ Probability
(Right to win is max)



⇒ Returns (G) discount factor ($0 \leq \gamma \leq 1$)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

 $\gamma = 0$ (immediate Reward) $\gamma = 1$ (long term) $\gamma = 0.5$

$$R_{t+1} + 0.5 \times R_{t+2} + (0.5)^2 \cdot \dots + (0.5)^3 + \dots$$

with value / high impact less value / less impact

⇒ Value fun (VF)

↳ help to predict Returns on a particular state (S)

Types

① State VF ($V_{\pi}(S)$)

↳ action on basis of state followed by policy

② Action VF $\rightarrow Q_{\pi}(a, S)$

↳ return on basis of state & action

③ $\rightarrow$ a) Optimal State VF $\rightarrow V^*(S)$ (max return possible)

$\rightarrow$ b) Optimal Action VF $Q^*(a, S)$

↳ max pos. return

⇒ MDP (Markov Decision Process)

Helps $\rightarrow$ How agent learn what action should take

Seq. of State, Action, Reward

may stop

(Tick Tac Toe)

may not stop

(Subway Surfer)

in never ending game

$\rightarrow$ Continuous

↓
terminate episode

$S_0 A_0 R_1, S_1 A_1 R_2, S_2 A_2 R_3, \dots$

MARKOV RULE :-

Nxt State depend on :

- ① Current State
- ② " Action

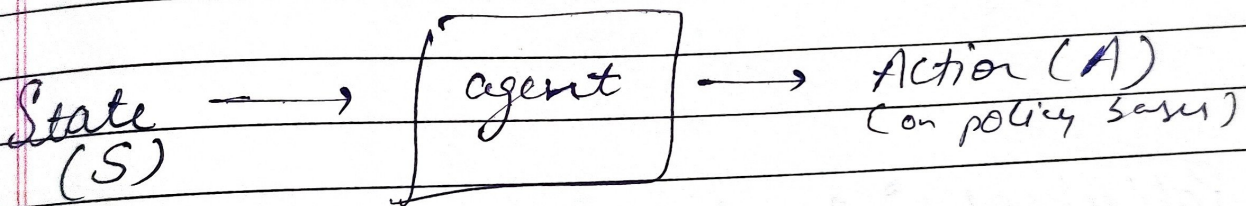
Imp. $\{$ NOT on PAST RESULT OR HISTORY.

$$S_{t+1} \leftarrow S_t, A_t \quad \checkmark$$

$$S_{t+1} \leftarrow S_{t-1}, A_{t-1} \quad \times$$

LIMITATION

- Not apply on all problem.
- NOT UNIVERSAL.



MDP uses by

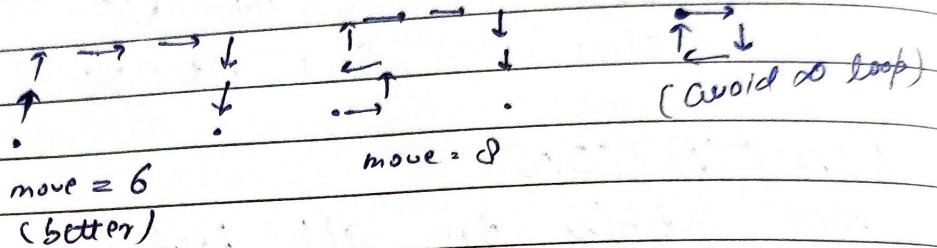
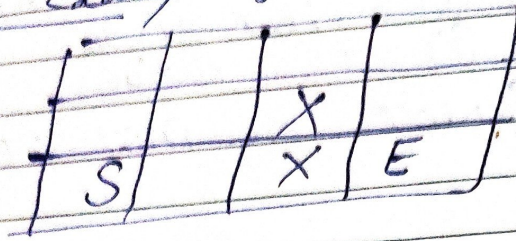
① Q-learning

② SARSA $\xrightarrow{\text{action}}$ State

$\downarrow$ $\downarrow$ $\swarrow$ $\searrow$
 state action Reward

③ Policy gradient

* Grid Example:



→ Allow fix move = 15 (To Avoid loop)

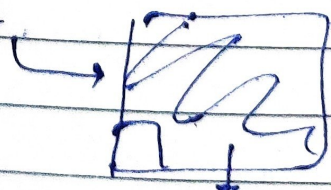
Policy Optimization

↳ world model

* Give model have a better idea how it can solve like we human can good solⁿ find by seeing our eyes.

$$P\left(\frac{s_{t+1}, a_{t+1}}{s_t, a_t}\right)$$

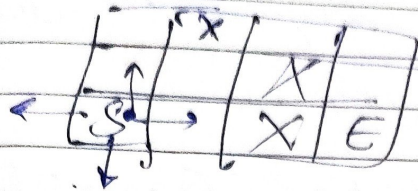
- ① Model Based RL → env ka full idea
- ② Model free RL



just as env nt
no idea

Policy opt.

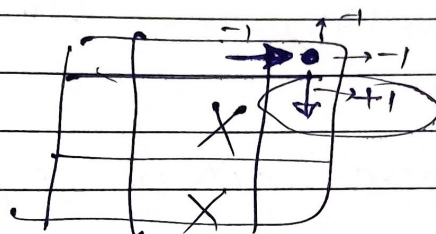
At first move $\rightarrow$ prob. of all move random (0, 1, 2, 3)
 $\downarrow$
reward = 0



At second move $\rightarrow$ better probs.

If take right step reward = 1

$\rightarrow$ wrong = -1 $\rightarrow$ can we predict better.



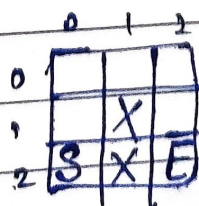
$\rightarrow$ Series of Action $\rightarrow$ better result
 $\rightarrow$ " " " " $\rightarrow$ worst "

increase for +ve result. this is called policy optimization.

Policy (π) $\rightarrow$ done using Policy Gradient Method

Policy

	S	A
$A(2,0)$	(2,0)	
U		1
D		-1
L		-1
R		-1



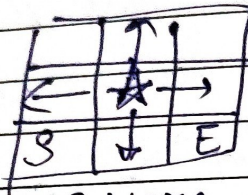
State VF Action VF

MY CHOICE
Date _____
Page _____
Action value $Q_{\pi}(s,a) >$ State value $V_{\pi}(s)$

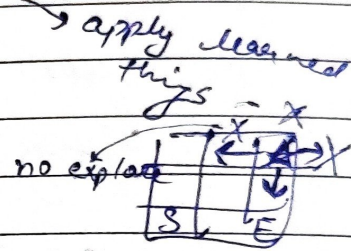
$$Q_{\pi}(s,a)$$

$\downarrow$
 $Q^*(s,a)$ (we pick only max return)
 $\rightarrow$ Greedy algo.

Exploration - Exploitation Trade Off



Explore
(new possibility)



$\rightarrow$ model can choose either explore or exploit.

Solⁿ :- (Q-learning)

$\Rightarrow$ ϵ -Greedy Strategy

$\downarrow$
 ϵ (prob. of explore)

$$\epsilon \approx 0.1$$

10% explore

90% exploit

$$0 \leq \epsilon \leq 1$$

random $0 \leq r \leq 1$

$r < \epsilon$ explore

$r > \epsilon$ exploit

Note :- $\epsilon \rightarrow$ gradually decrease

Beginner explore more

Expert exploit more

PART-2

REINFORCEMENT LEARNING

MY CHOICE

Date _____
Page _____

- How model learning
- Dynamic Programming
 - Monte Carlo
 - TD learning
- } less important. model free method.
 } model free
 } main
- ↓
in real world we have no full knowledge of env.

Monte-Carlo Method

→ model free

→ episodes → experienced samples.
(multiple possible path)

for Ex.
1st episode



Step	State	action	reward
1	2,0	↑	-1
2	1,0	↑	-1
3	0,0	→	-1
4	0,1	→	0

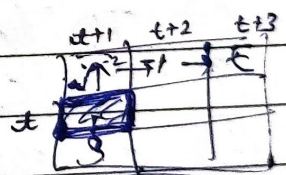
↓ stop.

Returns.

$$G_t = R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 R_{t+3}$$

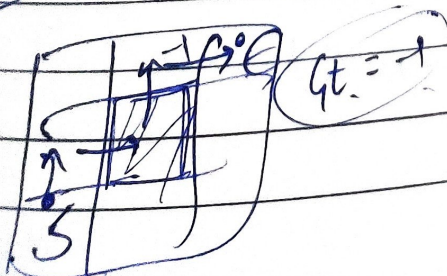
$$= -1 + (-1) + 0 = -2$$

$$G_t = -2$$



$$\gamma = 1$$

II nd episode



LIMITATION

- ① Long episodes → slow learning
- ② Continuous problem fail
- ③ Not step based learning

TD (Temporal Difference) Learning

↳ model free

→ Step base learning

1	2	3
3	1	2

$V(s)$

0	0	0
0	0	0
0	0	0

Step ①

update rules

reward

st. val. of cur. st.

$$V(s) = V(s) + \alpha \cdot [R + \gamma \cdot V(s') - V(s)]$$

learning rate

state value of next state

$\alpha = 0.5$ $\gamma = 1$

$$\begin{aligned} V(2,0) &= V(2,0) + 0.5 [-1 + 1 \cdot V(1,0) - V(2,0)] \\ &= 0 + 0.5 [-1 + 0 - 0] \\ &= -0.5 \end{aligned}$$

TD error

(Reduce it)

TD error ↓

Better model ↑

0	1	2
0	↑	
1	-1	
2	↑	

$V(s)$

-0.5	0	0
-0.5	0	0

TD Types

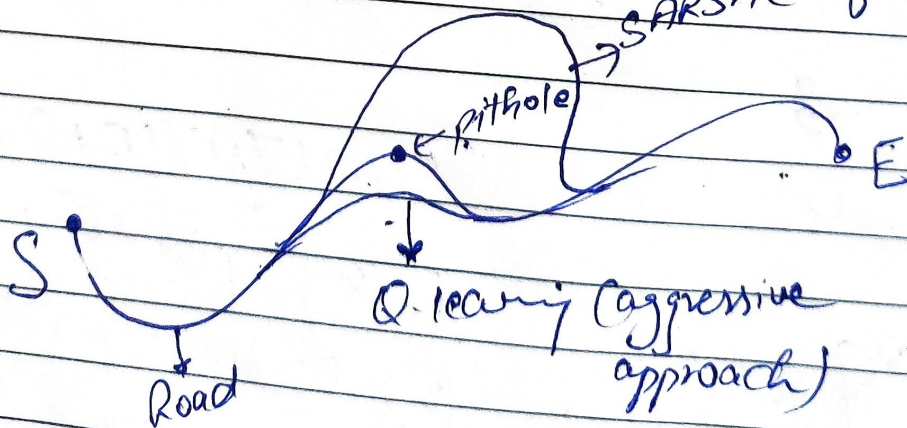
①

SARSA

②

Q-learning

SARSA (safer approach)



SARSA - (on-policy)

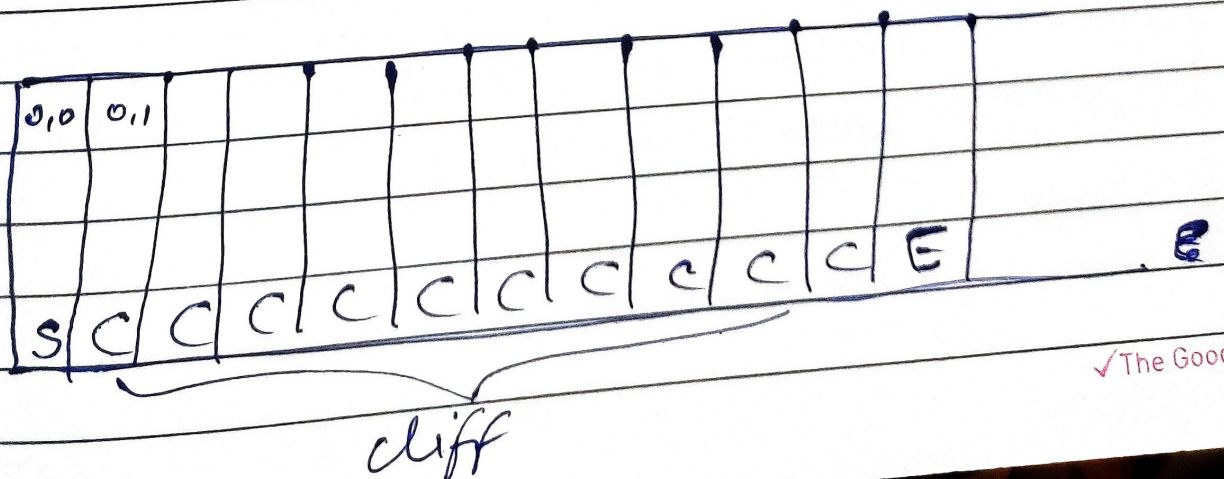
Steps

- ① Current State (s_t)
- ② " Action (A_t)
↳ use policy (ε-greedy) 10% exploration
90% exploit
- ③ Take action
Reward $\rightarrow R_{t+1}$
Next state $\rightarrow s_{t+1}$
- ④ Next action (A_{t+1}) $\rightarrow$ policy (s_{t+1}, A_{t+1})
- ⑤ learning step by update Q-value
↳ $Q(s, a) = Q(s, a) + \alpha [R + \gamma \cdot Q(s', a') - Q(s, a)]$

Q-learning - (off-policy)

- ① s_t
 - ② $A_t \rightarrow$ ε-greedy
 - ③ R_{t+1}, s_{t+1}
 - ④ $A_{t+1} \Rightarrow$ on basis of Q-value
 - ⑤ update Q-value.
- ↑ optimal Q-value
- $$Q(s, a) = Q(s, a) + \alpha [R + \gamma \max_{a'} (Q(s', a')) - Q(s, a)]$$

Cliff Walking Problem 4x12 grid



env: 4x12 grid
Action: 0: $\downarrow$, 1: $\leftarrow$, 2: $\rightarrow$, 3: $\uparrow$ $\{0, 3\}$

Reward:
normal cell $\rightarrow -1$
target " $\rightarrow 0$
cliff $\rightarrow -100$

State: $4 * 12 = 48$ cells

$$\boxed{\text{State} = \text{row} * 12 + \text{col}}$$

for $0,0 \Rightarrow 0 * 12 + 0 = 0$
 $0,1 \Rightarrow 0 * 12 + 1 = 1$

0	1	2	3	4	...
12	.	.	.	.	.
S					E

$S \Rightarrow (3,0) \Rightarrow 3 * 12 + 0 \Rightarrow 36.$

11 Gymnasium environment

11 code

Deep RL = DL + RL

⇒ Deep Q-network

- ↳ Continuous + ∞ states.
- use when Q-table fail
- state $\rightarrow$ fix $\rightarrow$ actions
(input network) (returns Q-values)

Q-learning $\Rightarrow Q(s, a)$

DQN $\Rightarrow Q(s, a; \theta)$
parameters

⇒ Problems

- ↳ ① Correlated samples
- ② Moving Targets

① Correlated Samples

NN best train when I/P is independent

experience pass to NN

(S, A, R, S', T)

↓
Terminated

① Sequential - $S_1 \rightarrow S_2 \rightarrow \dots$

② Correlated - Before S_2
 S_1 need to complete

E1: $S_1 \rightarrow A_1 \rightarrow R_2 \rightarrow S_2$

E2: $S_2 \rightarrow A_2 \rightarrow R_3 \rightarrow S_3$

Solⁿ: ⇒ Experience Replay

↳ create memory (DD) buffer

↓
Store exp. → randomly sample minibatches

$E_1 \rightarrow E_1 \rightarrow \dots$ X (Fixed)

$E_1 \rightarrow E_2 \rightarrow E_5$ ✓ (better training b/c now not in seq.)

② Moving Target

$$Q(s,a) \leftarrow \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Q-learning

Target (y) = $r + \gamma \max(\bar{Q}(s',a;\theta))$
 predict Q-value = $Q(s',a;\theta) \Rightarrow \hat{y}$

Not Fixed.

↓
result in unstable training

$$\text{loss} = y - \hat{y}$$

↓
ex. snake change its own tail

Solⁿ ⇒ Target Network

- ① Policy Network
- ② Target n/w (old copy of policy n/w)

Flappy Bird